

Practical Session 8

Task 1

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
j_rakshitha@RAKSHITHA: ~  
j_rakshitha@RAKSHITHA:~$ valgrind --version  
valgrind-3.26.0  
j_rakshitha@RAKSHITHA:~$ nano session8_task1_memory.c  
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra session8_task1_memory.c -o session8_task1_memory  
j_rakshitha@RAKSHITHA:~$ ./session8_task1_memory  
Practical Session 8 - Task 1  
=====
```

```
malloc() values: 10 20 30 40 50  
calloc() values: 0 0 0 0 0  
After realloc(), values: 10 20 30 40 50 60 70 80 90 100  
  
Intentional memory leak created.  
Program finished.  
j_rakshitha@RAKSHITHA:~$ valgrind --leak-check=full ./session8_task1_memory  
==1334== Memcheck, a memory error detector  
==1334== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.  
==1334== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info  
==1334== Command: ./session8_task1_memory  
==1334==  
Practical Session 8 - Task 1  
=====
```

```
malloc() values: 10 20 30 40 50  
calloc() values: 0 0 0 0 0  
After realloc(), values: 10 20 30 40 50 60 70 80 90 100  
  
Intentional memory leak created.  
Program finished.  
==1334==  
==1334== HEAP SUMMARY:  
==1334==    in use at exit: 400 bytes in 1 blocks  
==1334==    total heap usage: 5 allocs, 4 frees, 1,504 bytes allocated  
==1334==  
==1334== 400 bytes in 1 blocks are definitely lost in loss record 1 of 1  
==1334==    at 0x4850858: malloc (vg_replace_malloc.c:447)  
==1334==    by 0x40014A9: main (in /home/j_rakshitha/session8_task1_memory)  
==1334==  
==1334== LEAK SUMMARY:  
==1334==    definitely lost: 400 bytes in 1 blocks  
==1334==    indirectly lost: 0 bytes in 0 blocks  
==1334==    possibly lost: 0 bytes in 0 blocks  
==1334==    still reachable: 0 bytes in 0 blocks  
==1334==    suppressed: 0 bytes in 0 blocks  
==1334==  
==1334== For lists of detected and suppressed errors, rerun with: -s  
==1334== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)  
j_rakshitha@RAKSHITHA:~$
```

```
j_rakshitha@RAKSHITHA: ~  ×  +  ∨
GNU nano 8.7.1 session8_task1_memory.c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    printf("Practical Session 8 - Task 1\n");
    printf("=====\n");

    /* malloc() */
    int *a = malloc(5 * sizeof(int));

    if (a == NULL)
    {
        perror("malloc");
        return 1;
    }

    for (int i = 0; i < 5; i++)
        a[i] = (i + 1) * 10;

    printf("\nmalloc() values: ");
    for (int i = 0; i < 5; i++)
        printf("%d ", a[i]);
    printf("\n");

    /* calloc() */
    int *b = calloc(5, sizeof(int));

    if (b == NULL)
    {
        perror("calloc");
        free(a);
        return 1;
    }

    printf("calloc() values: ");
    for (int i = 0; i < 5; i++)
        printf("%d ", b[i]);
    printf("\n");

    /* realloc() */
    a = realloc(a, 10 * sizeof(int));

    if (a == NULL)
    {
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session8_task1_memory.c

printf("calloc() values: ");
for (int i = 0; i < 5; i++)
    printf("%d ", b[i]);
printf("\n");

/* realloc() */
a = realloc(a, 10 * sizeof(int));

if (a == NULL)
{
    perror("realloc");
    free(b);
    return 1;
}

for (int i = 5; i < 10; i++)
    a[i] = (i + 1) * 10;

printf("After realloc(), values: ");
for (int i = 0; i < 10; i++)
    printf("%d ", a[i]);
printf("\n");

/* Free the allocations that we no longer need */
free(a);
free(b);

/* Intentional memory leak */
int *leak = malloc(100 * sizeof(int));

if (leak == NULL)
{
    perror("malloc");
    return 1;
}

leak[0] = 999;

printf("\nIntentional memory leak created.\n");
printf("Program finished.\n");

return 0;
}
|
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location
```

Task 2

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```
j_rakshitha@RAKSHITHA:~$ nano session8_task2_cow.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra session8_task2_cow.c -o ses
sion8_task2_cow
j_rakshitha@RAKSHITHA:~$ ./session8_task2_cow
Practical Session 8 - Task 2
=====

Before fork() - Parent memory:
VmRSS:      11932 kB

Parent process. PID: 1405

Child process created. PID: 1406

Child memory before modification:
VmRSS:      11244 kB

Child memory after modification:
VmRSS:      11244 kB

Child modified the memory.
Copy-on-Write causes modified pages to get private copies.

Parent memory after child finishes:
VmRSS:      12060 kB
j_rakshitha@RAKSHITHA:~$ |
```

j_rakshitha@RAKSHITHA: ~



GNU nano 8.7.1

session8_task2_cow.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define SIZE (10 * 1024 * 1024)

void show_memory(const char *label)
{
    FILE *file;
    char line[256];

    printf("\n%s\n", label);

    file = fopen("/proc/self/status", "r");

    if (file == NULL)
    {
        perror("fopen");
        return;
    }

    while (fgets(line, sizeof(line), file))
    {
        if (strncmp(line, "VmRSS:", 6) == 0)
        {
            printf("%s", line);
            break;
        }
    }

    fclose(file);
}

int main(void)
{
    char *memory;

    printf("Practical Session 8 - Task 2\n");
    printf("=====\n");

    memory = malloc(SIZE);

    if (memory == NULL)
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session8_task2_cow.c
    perror("malloc");
    return 1;
}

/* Touch every page so the memory is actually allocated */
for (int i = 0; i < SIZE; i += 4096)
{
    memory[i] = 'A';
}

show_memory("Before fork() - Parent memory:");

pid_t pid = fork();

if (pid < 0)
{
    perror("fork");
    free(memory);
    return 1;
}

if (pid == 0)
{
    printf("\nChild process created. PID: %d\n", getpid());

    show_memory("Child memory before modification:");

    /* Modify the memory in the child */
    for (int i = 0; i < SIZE; i += 4096)
    {
        memory[i] = 'B';
    }

    show_memory("Child memory after modification:");

    printf("\nChild modified the memory.\n");
    printf("Copy-on-Write causes modified pages to get private copies.\n");

    free(memory);
    exit(0);
}
else
{
    printf("\nParent process. PID: %d\n", getpid());

    printf("\nParent process. PID: %d\n", getpid());

    wait(NULL);

    show_memory("Parent memory after child finishes:");

    free(memory);
}

return 0;
}
```